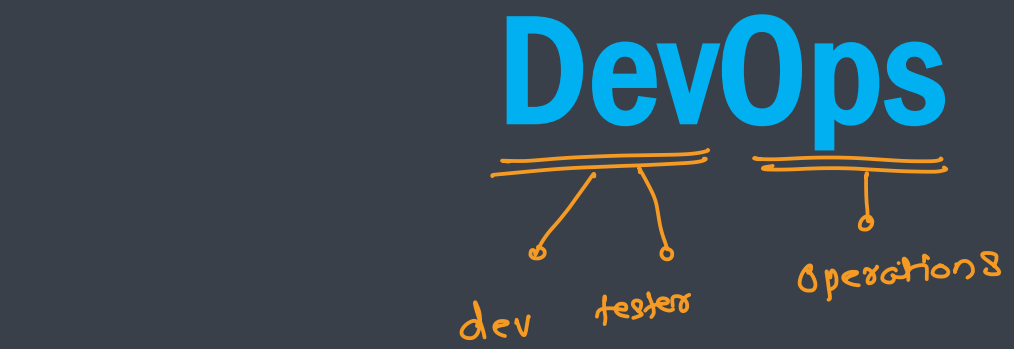
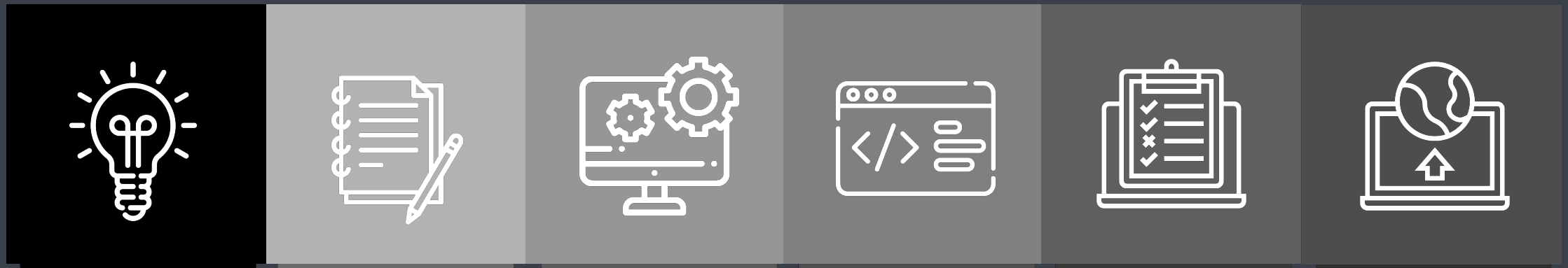

DevOps
dev
testers
operations

Software Development Lifecycle



PLANNING

Talk to customer and understand the requirements

DEFINING

Define the requirements and stick to them

Software Requirements Specifications (SRS)

DESIGNING

Design the solution with right approach

BUILDING

Development following guidelines

TESTING

Make sure that your code is working

manual automated

DEPLOYMENT

Make your app available for rest of the world

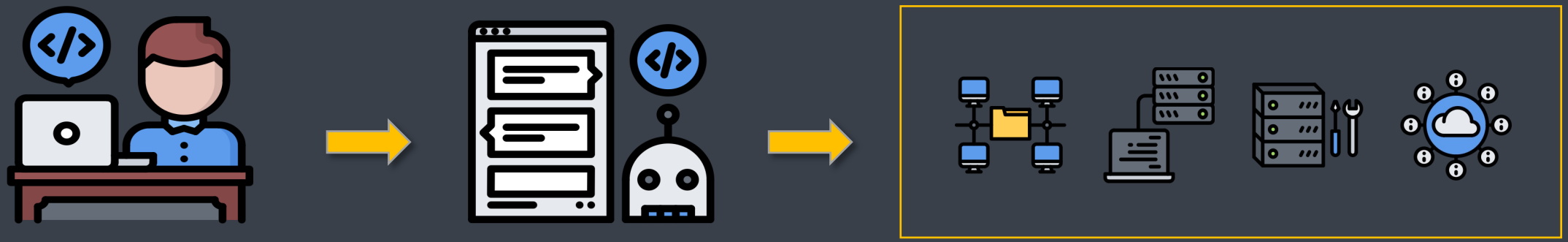
open to world

cloud

Waterfall Model

- Requirement Specification
- System Design
- Design Implementation
- Verification & Testing
- System Deployment
- Software Maintenance

Entities involved



Developer

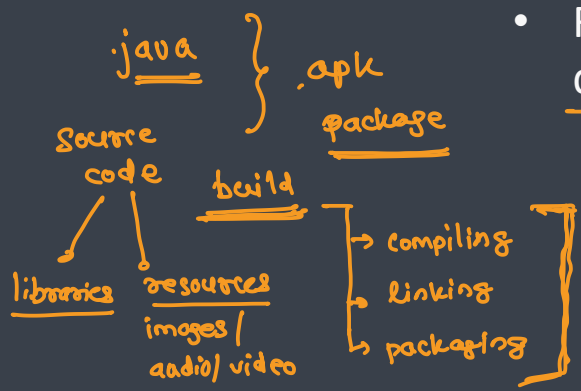
Testers

Operations Team

Responsibilities

Developers and Testers

- Developers
  - Develop the application
  - Package the application
  - Fix the bugs
  - Maintain the application
- Testers
  - Thoroughly test the application manually or using test automation
  - Report the bugs to the developer



Operations Team

- Make all the necessary resources ready
- Deploy the application
- Maintain multiple environments
- Continuously monitor the application
- Manage the resources

Challenges

Developers and Testers

- The process is slow
- The pressure to work on the newer features and fix the older code
- Not flexible

Gap

Operations Team

- Uptime
- Configure the huge infrastructure
- Diagnose and fix the issue

monitoring

python → 3.11 ← E ✓

latest → features

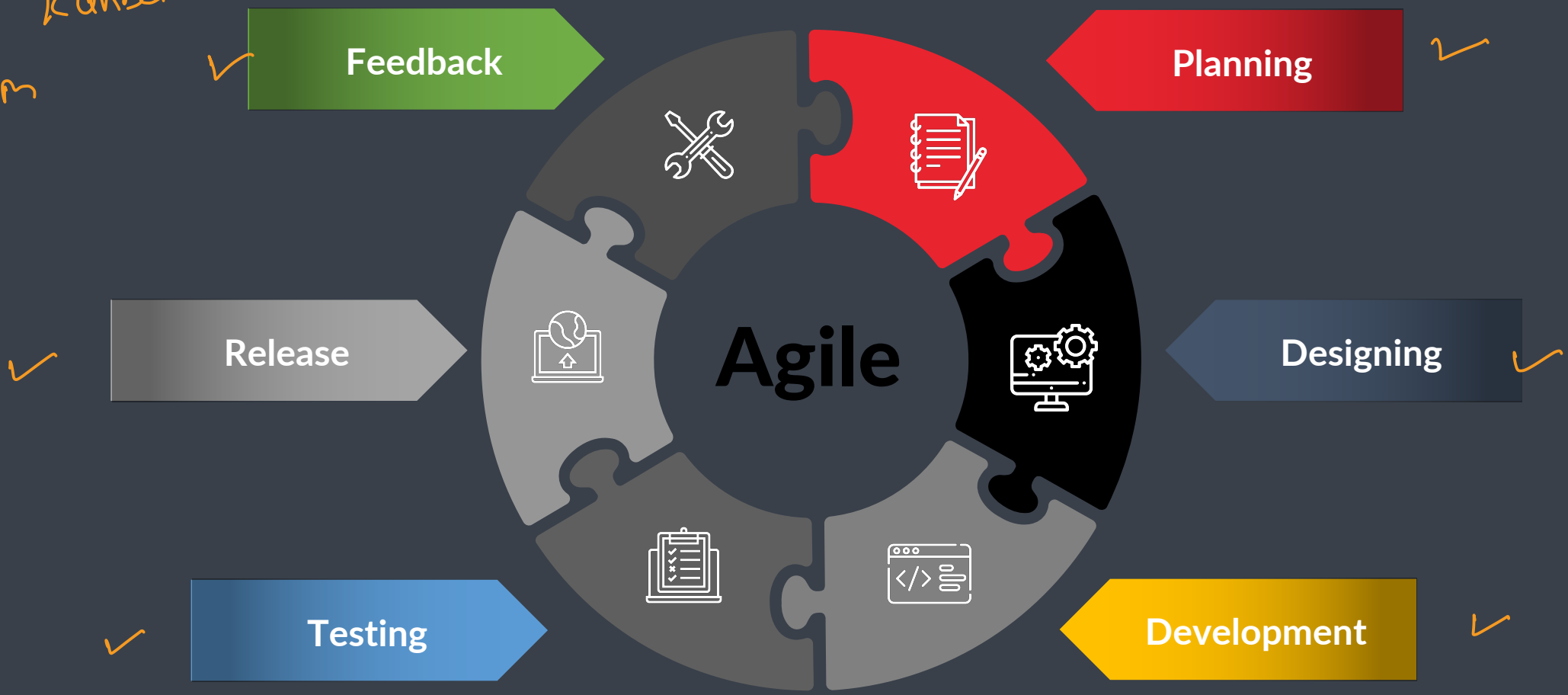
Stable version

3.8 → ✗

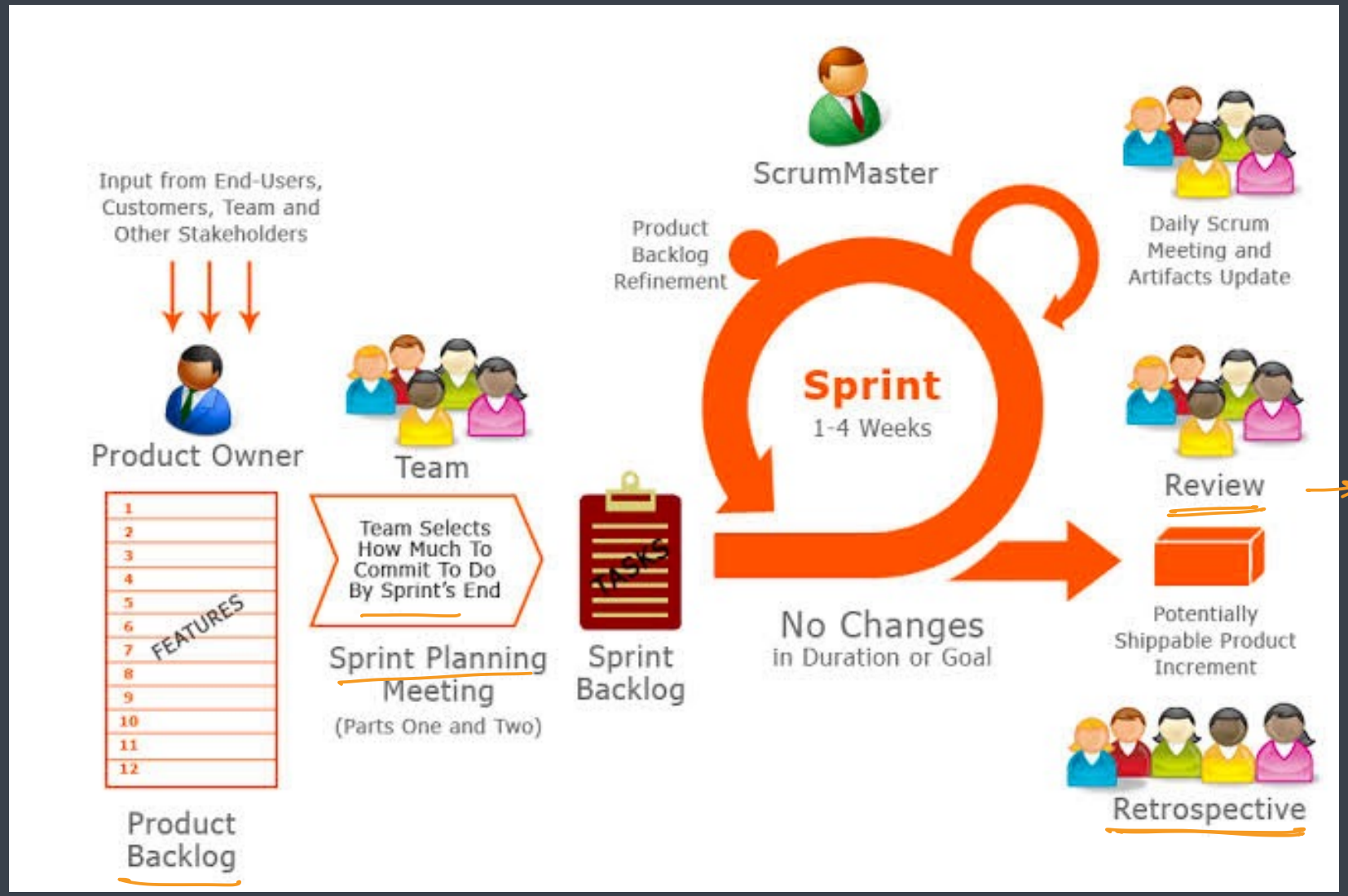
Agile Development

Scrum

Kanban



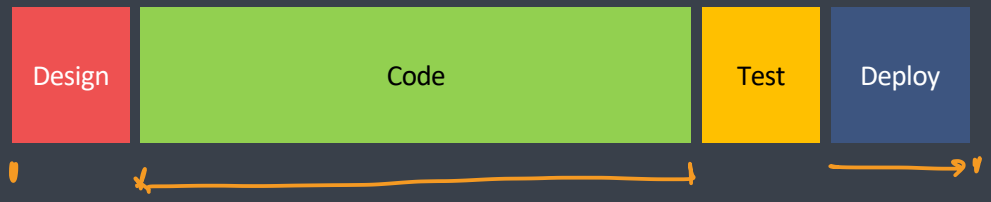
Scrum Process



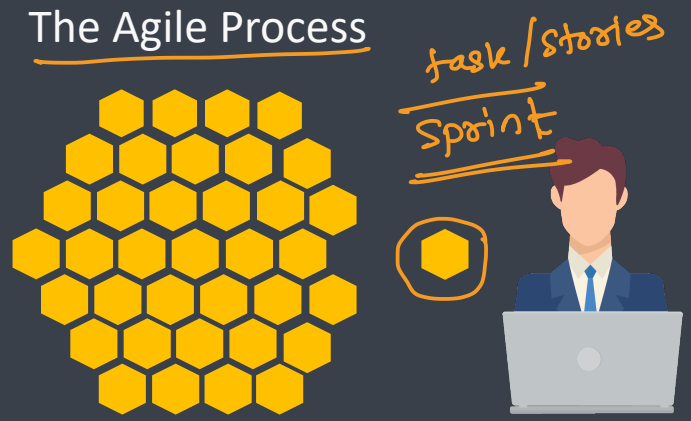
Waterfall Vs Agile

The Waterfall Process

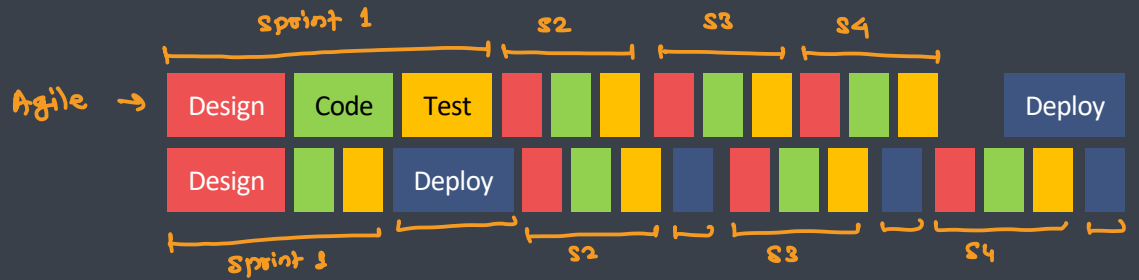
This project has got so big.
I am not sure I will be able to deliver it!



The Agile Process



It is so much better delivering
this project in bite-sized sections



Problems

- Managing and tracking changes in the code is difficult
- Incremental builds are difficult to manage, test and deploy
- Manual testing and deployment of various components/modules takes a lot of time
- Ensuring consistency, adaptability and scalability across environments is very difficult task
- Environment dependencies makes the project behave differently in different environments

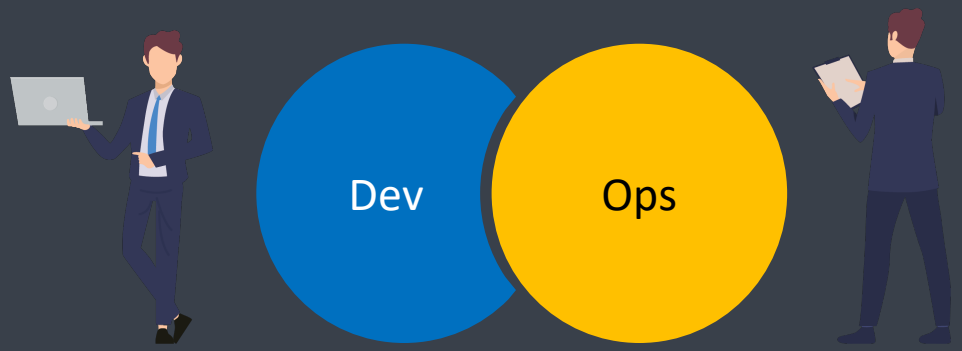
Solutions to the problem

- Managing and tracking changes in the code is difficult: **SCM tools**
- Incremental builds are difficult to manage, test and deploy: **Jenkins** [CI/CD tools]
- Manual testing and deployment of various components/modules takes a lot of time: **Selenium** [test automation]
- Ensuring consistency, adaptability and scalability across environments is very difficult task: **Puppet** [continuous conf tools] → [Infrastructure As a code] → IaC
- Environment dependencies makes the project behave differently in different environments: **Docker** [Containerization]

What is DevOps ?

dev testing

- DevOps is a combination of two words development and operations
- Promotes collaboration between Development and Operations Team to deploy code to production faster in an automated & repeatable way
- DevOps helps to increases an organization's speed to deliver applications and services → automation
- It allows organizations to serve their customers better and compete more strongly in the market
- Can be defined as an alignment of development and IT operations with better communication and collaboration
- DevOps is not a goal but a never-ending process of continuous improvement
- It integrates Development and Operations teams
- It improves collaboration and productivity by
  - Automating infrastructure → Configuration tools
  - Automating workflow → CI/CD tools
  - Continuously measuring application performance → monitoring tools



Why DevOps is Needed?

- Before DevOps, the development and operation team worked in complete isolation
- Testing and Deployment were isolated activities done after design-build. Hence they consumed more time than actual build cycles.
- Without using DevOps, team members are spending a large amount of their time in testing, deploying, and designing instead of building the project.
- Manual code deployment leads to human errors in production → automation
- Coding & operation teams have their separate timelines and are not in synch causing further delays

Common misunderstanding

- DevOps is not a role, person or organization
- DevOps is not a separate team
- DevOps is not a product or a tool
- DevOps is not just writing scripts or implementing tools

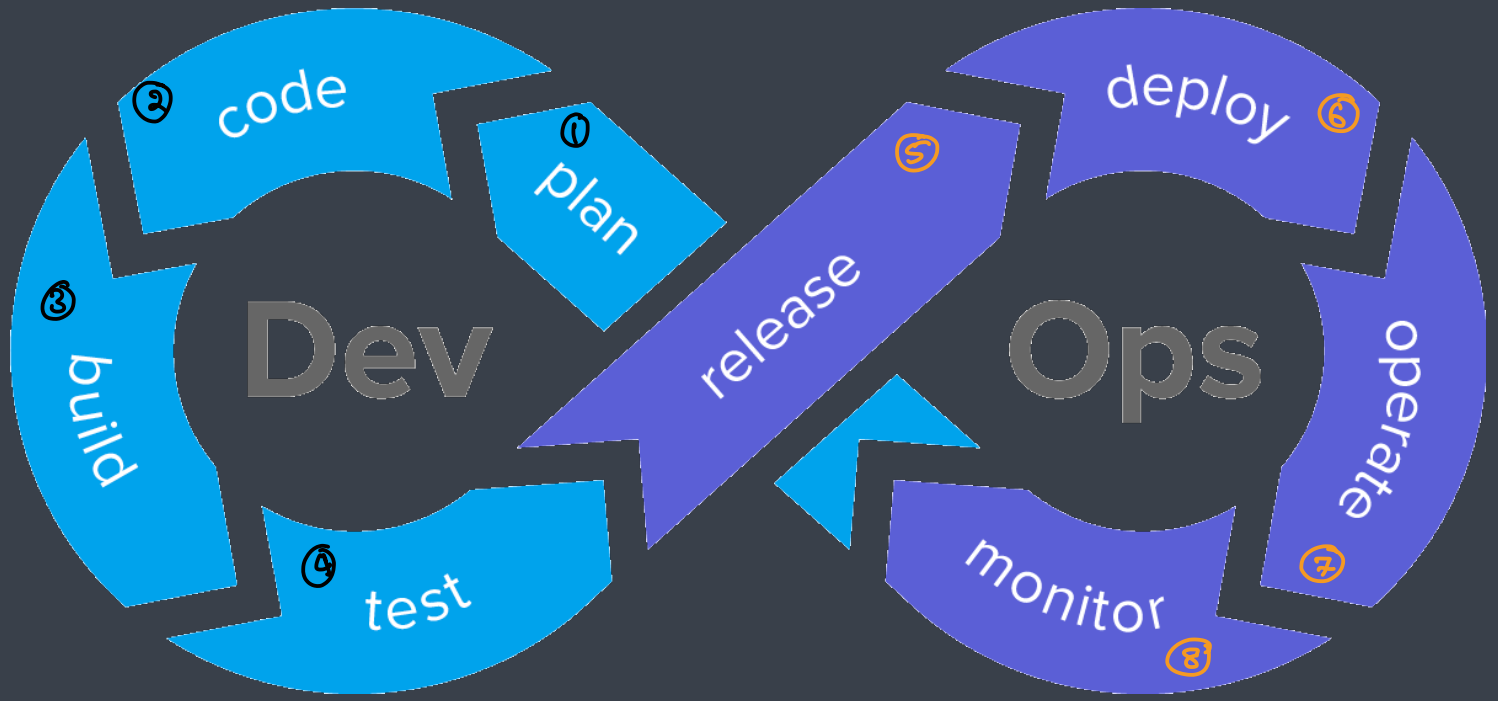
x → operations team

continuous improvements

Reasons to use DevOps

- **Predictability**
  - DevOps offers significantly lower failure rate of new releases
- **Reproducibility**
  - Version everything so that earlier version can be restored anytime
- **Maintainability**
  - Effortless process of recovery in the event of a new release crashing or disabling the current system
- **Time to market**
  - DevOps reduces the time to market up to 50% through streamlined software delivery
  - This is particularly the case for digital and mobile applications
- **Greater Quality**
  - DevOps helps the team to provide improved quality of application development as it incorporates infrastructure issues
- **Reduced Risk**
  - DevOps incorporates security aspects in the software delivery lifecycle. It helps in reduction of defects across the lifecycle
- **Resiliency**
  - The Operational state of the software system is more stable, secure, and changes are auditable

DevOps Lifecycle



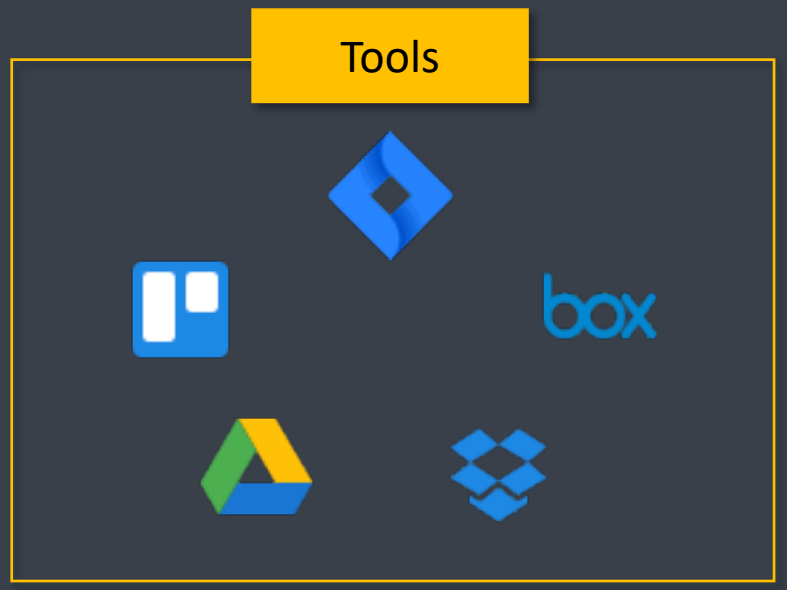
DevOps Lifecycle - Plan

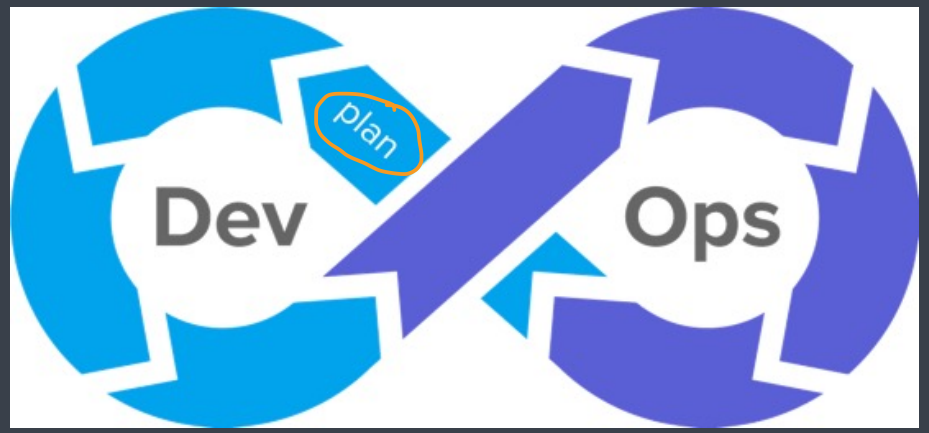
- First stage of DevOps lifecycle where you plan, track, visualize and summarize your project before you start working on it

- notepad / editor
- dropbox, google drive, one drive
- google sheet, excel

} ad hoc

agile tools / scrum tools
- jira **
- orange scrum





DevOps Lifecycle - Code : programming

- Second stage where developer writes the code using favorite programming language

→ tool chain → compiler, interpreter, linker, assembler, debugger etc
SDK

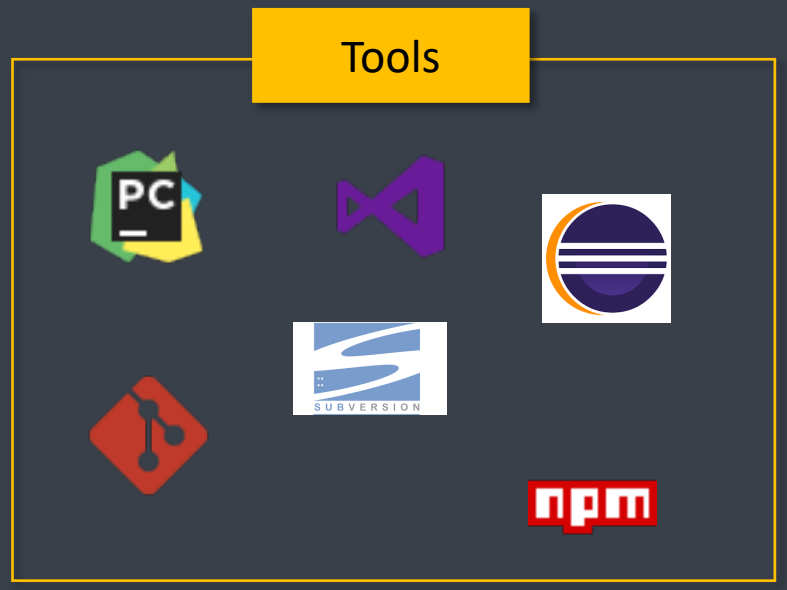
→ languages → C, C++, C#, java, python etc.

→ editors → vim, vscode, atom, sublime

IDE → Pycharm, Jupyter, Spyder, Eclipse, IDEA, webstorm, clion, Android studio, PHPstorm etc

package managers → pip (python), npm (JS), cocoapods (swift), maven (java)

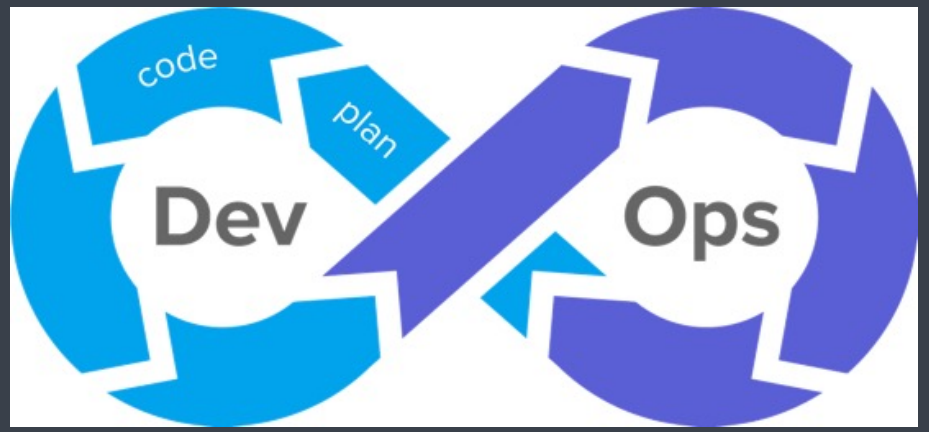
Tools



SCM → git, bitkeeper
dvcs

SVN, cvs, bazaar
cvcs

sourcesafe,
Lvcs



DevOps Lifecycle -Build

- Integrating the required libraries
- Compiling the source code
- Create deployable packages

- build tools → ant, maven, gradle *

scripts → make, custom scripting

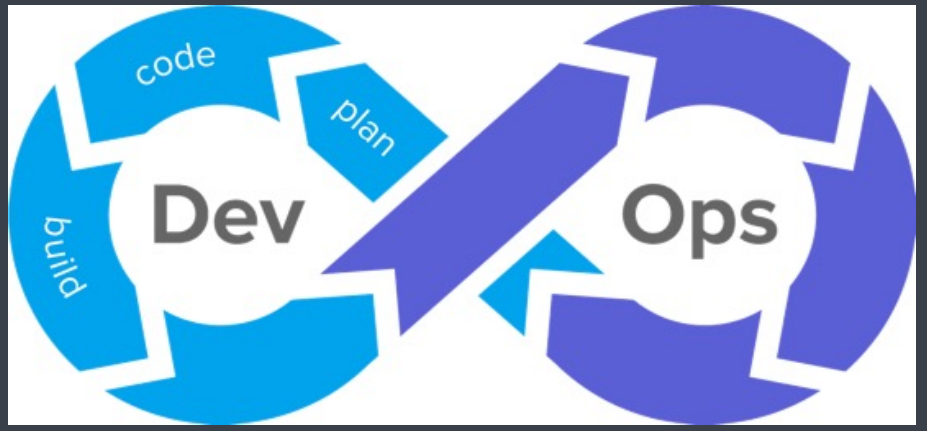
packaging → build = compile + dependencies + document ⇒ package

↳ web → webpack
android → .apk
ios → .ipa
windows → .msi
mac OS → .dmg

Linux
rpm deb
dpkg

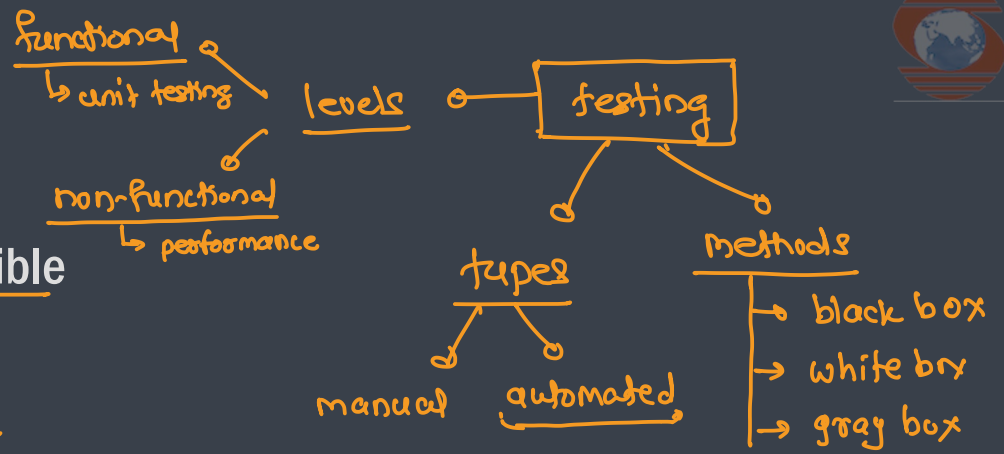
Tools

maven



DevOps Lifecycle - Test

- Process of executing automated tests
- The goal here is to get the feedback about the changes as quickly as possible

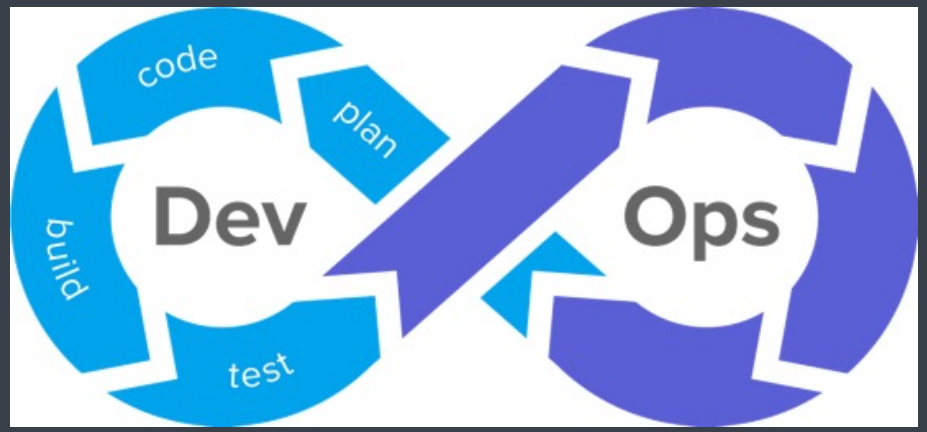


→ unit testing → PyUnit, JUnit, NUnit, Jasmine, Jest

→ performance testing tools → JMeter, stressTester, LoadTester
WinRunner

test automation tools → Selenium, cypress, TestNG

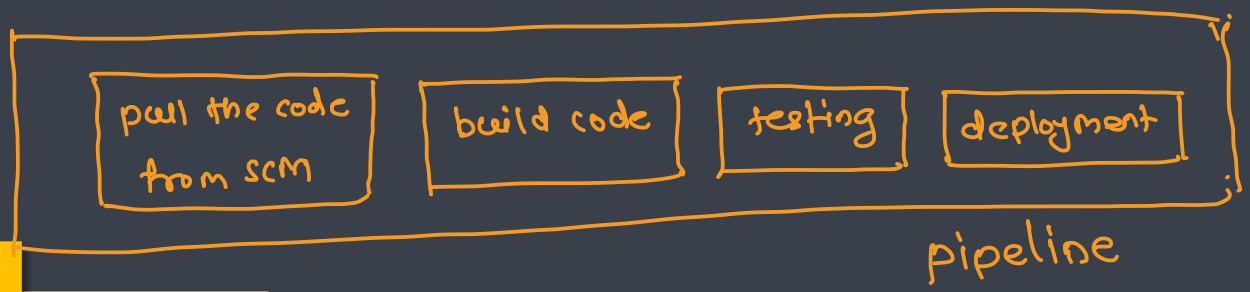
Tools

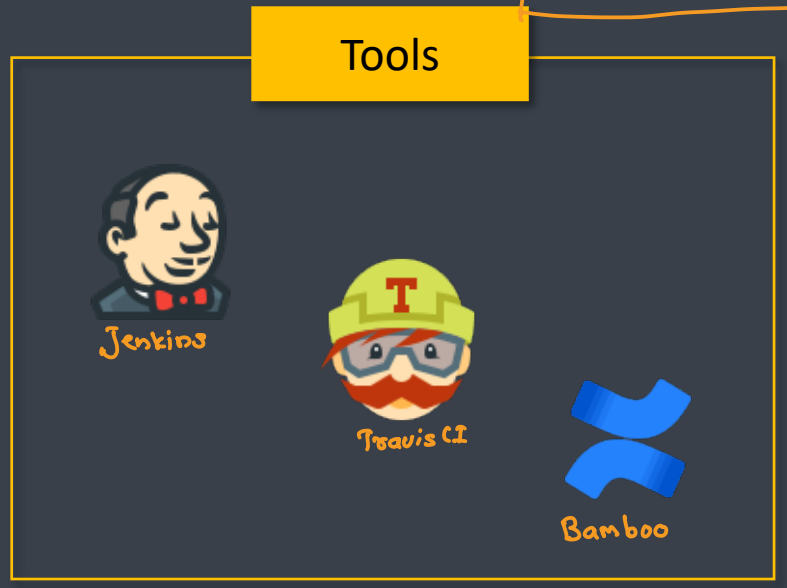


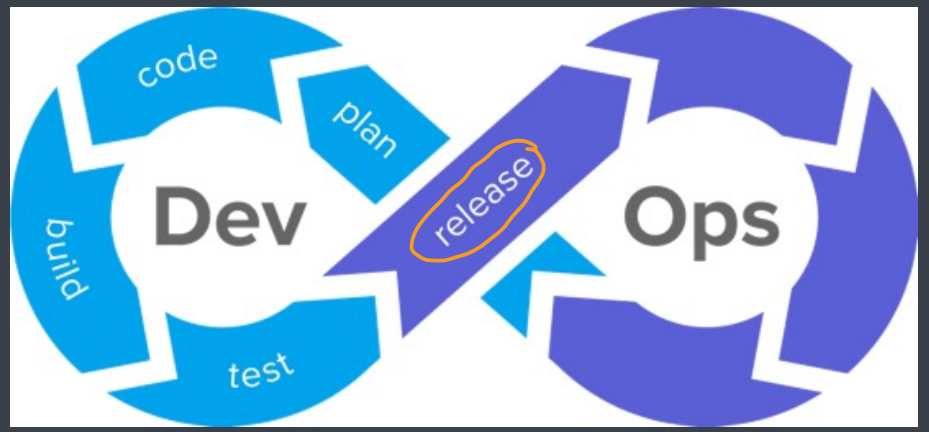
DevOps Lifecycle - Release

- This phase helps to integrate code into a shared repository using which you can detect and locate errors quickly and easily

CI/CD tools → Jenkins, Travis CI, Bamboo, GitLab CI, GitHub Actions, ArgoCD







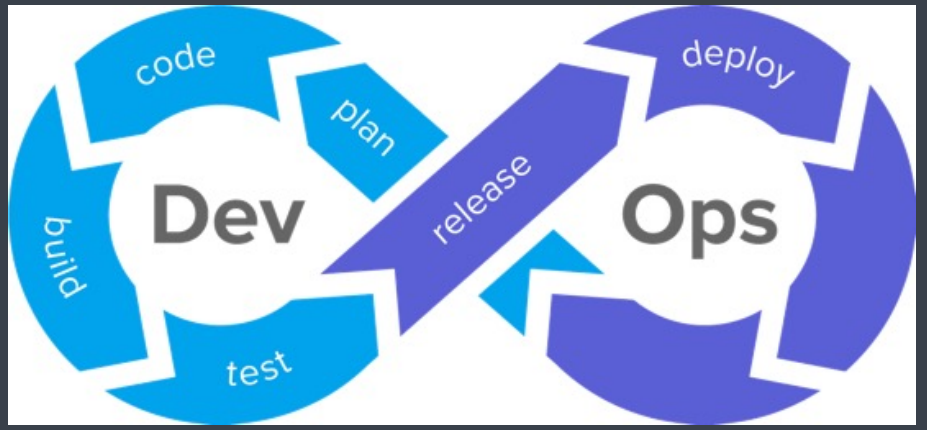
DevOps Lifecycle - Deploy

- Manage and maintain development and deployment of software systems and server in any computational environment

deployment

- traditional deployment : using physical machine
- virtualized deployment : using virtual machines
  - type I : vmware ESXi
  - type II : vmware, vBox, EC2
- containerized deployment : using containers → docker, podman, rkt, LXC, LXD

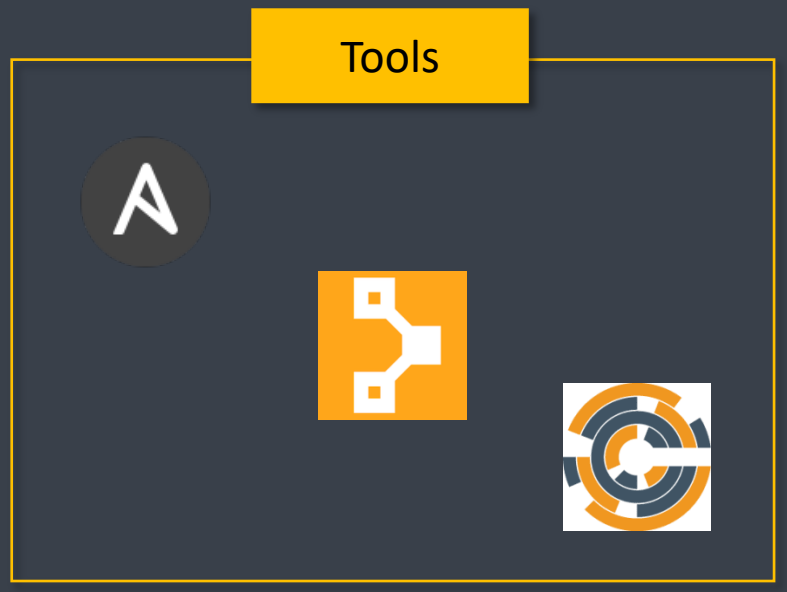
Tools

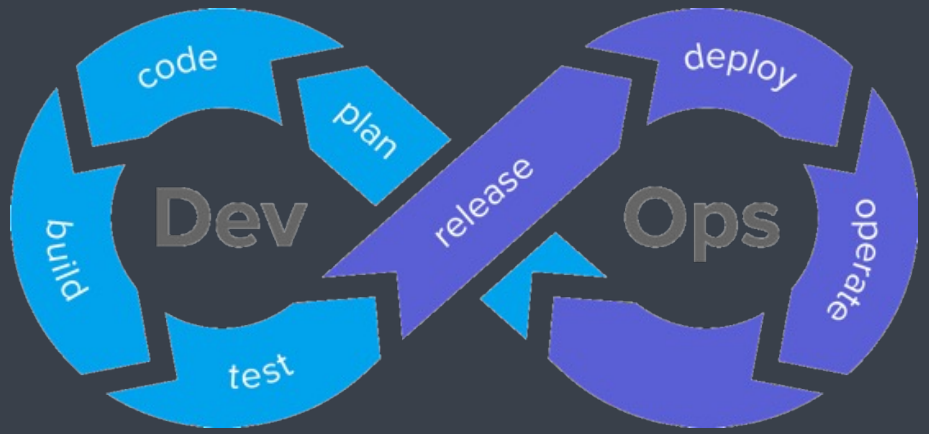


DevOps Lifecycle - Operate → IaC

- This stage where the updated system gets operated

continuous configuration tools → puppet, chef, ansible, saltStack





DevOps Lifecycle - Monitor

- It ensures that the application is performing as expected and the environment is stable
- It quickly determines when a service is unavailable and understand the underlying causes

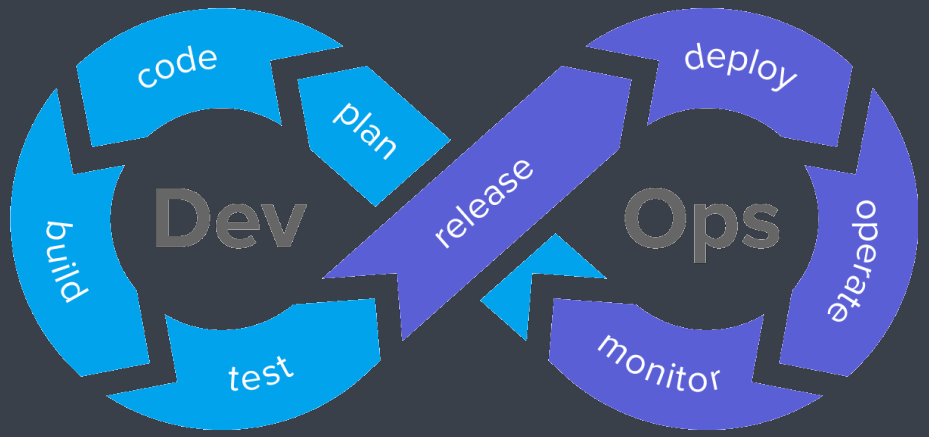
continuous monitoring tools :→ nagios, splunk, Datadog,

Tools

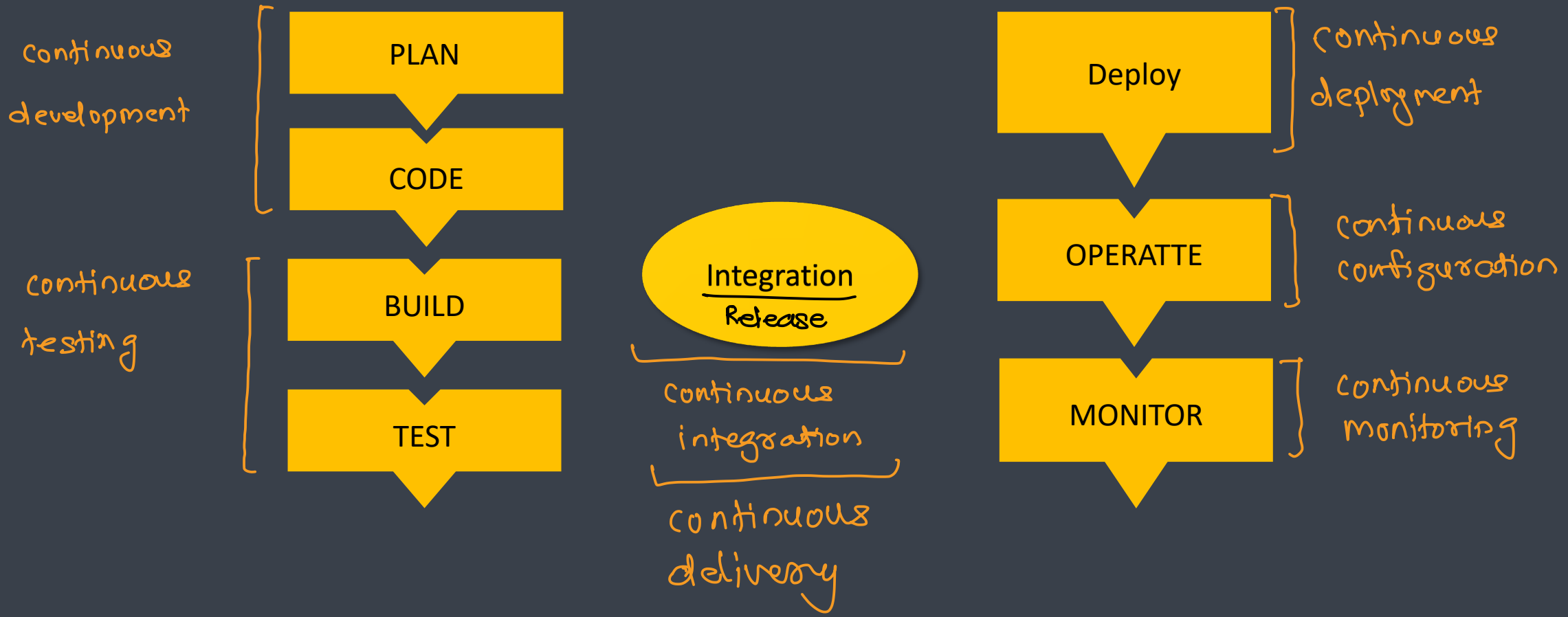
splunk>

DATADOG

Nagios®



DevOps Terminologies



Responsibilities of DevOps Engineer

Linux + win administration

- Be an excellent sysadmin

EC2 → cloud / on-prem

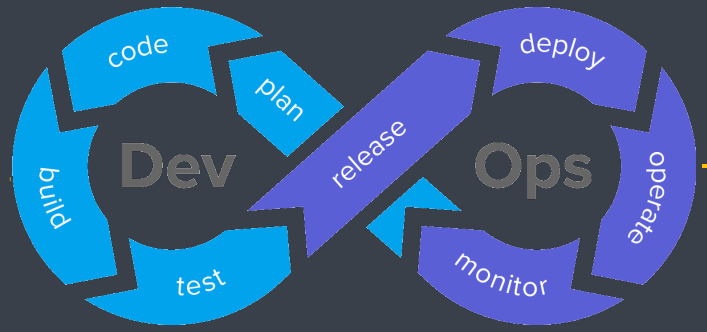
- Deploy Virtualization

networking

- Hands-on experience in network and storage

python

- Introduction to coding



- Soft skills ??

puppet

- Automation tools

selenium

- Software Testing knowledge

security

- IT security

Skills of a DevOps Engineer

Skills	Description
Tools	• Version Control – Git/SVN • Continuous Integration – Jenkins • Virtualization / Containerization – Docker/Kubernetes • Configuration Management – Puppet/Chef/Ansible • Monitoring – Nagios/Splunk
Network Skills	• General Networking Skills • Maintaining connections/Port Forwarding
Other Skills	• Cloud: AWS/Azure/GCP • Soft Skills • People management skill